Day 11 (Oct 30): Revision Sheet

Python conditions, lists and tuples, slicing | Git branch and merge | Excel $ and boundaries. Fresh recall from memory, no notes.

1. KEY IDEAS

Python: conditions

- if/elif/else: Python runs only the **first True** branch and skips the rest. A line at the left edge after the chain runs every time.
- Two separate ifs are each checked on their own. Not nested: nested means one if sits **inside** another if's block.
- An **else pairs with the nearest if above it at the same indentation**, not the last if. If the outer if is False, the inner condition is never evaluated.
- **and** needs two complete comparisons: marks >= 30 and marks < 40. A bare '< 40' is a SyntaxError (raised before the program runs). Python also allows 30 <= marks < 40; Excel does not.

Python: lists and tuples (took longer, more depth)

- In-place methods return **None**: sort, append, insert, remove, reverse. pop() returns the removed item. sorted() returns a new list and leaves the original alone.
- (5) is an int; the comma makes the tuple: (5,) or 5, . An empty tuple () needs no comma.
- A tuple stores references. You **cannot make a slot point to a different object** (TypeError), but a mutable object inside it, like a list, can change. t.append(3) fails with AttributeError; t[1].append(3) works.
- Slices: stop is excluded; the **sign of the step** sets the direction; slices clip out-of-range numbers (no error), but a single index out of range raises IndexError. If the step direction cannot reach the stop, the result is an empty list.

Git: branch and merge

- **Fast-forward**: main has no new commits since the branch was made, so Git just moves the main label. Straight line, no new commit.
- **Merge commit**: both branches have new commits (diverged). New commit with **2 parents**; Git prints "Merge made by the 'ort' strategy." This can happen with no conflict.
- **Conflict**: same lines of the same file changed on both branches. Fix: open the file, remove all 3 marker lines, write the final text, git add, git commit.
- Parents: first commit 0, ordinary commit 1, merge commit 2. Each commit stores the hash of the commit it was built on.

Excel

- Missing $ gives a valid formula, wrong numbers, **no error**. Put $ on the part that stays. Boundary rows: >= vs > comes from the wording (at least = >=, more than = >).

2. EXAMPLES TO KNOW

```
marks = 40
if marks > 40:        # False
    print("A")
elif marks >= 40:     # True, runs, chain stops
    print("B")
else:
    print("C")
print("Done")         # left edge, always runs
# output: B then Done
```

```
t = (1, [2, 3])
t[1].append(4)   # ok  -> (1, [2, 3, 4])
t[1] = [9]       # TypeError: 'tuple' object does not support item assignment
```

```
nums = [10, 20, 30, 40, 50]
nums[1:4]    -> [20, 30, 40]       nums[::-1]   -> [50, 40, 30, 20, 10]
nums[2:100]  -> [30, 40, 50]       nums[1:4:-1] -> []
nums[-1:-4:-1] -> [50, 40, 30]     nums[5]      -> IndexError
```

3. MY MISTAKES AND THE FIX

What I said or did	The fix
Gave output with no reason, or only '?'	Always give output AND reason AND value. Trace the exact code line by line.
Called two ifs 'nested'; called an if 'inner loop'	Nested = inside another block. An if decides once, a loop repeats.
Else belongs to the 'last if'	Else pairs with the nearest if above it at the same indentation.
'Stop is optional' as the reason nums[2:100] has no error	Slices clip out-of-range numbers. Only a single index raises IndexError.
x = nums.sort() then 'x same as nums'	x is None (capital N). sort() sorts in place and returns None.
Read t[1] = [9] as 't1 = 9'	Read the exact line in front of you.
'Can't change a list inside a tuple'	The list can change; the slot cannot point to a new object.
num.append(3) on a tuple	AttributeError: 'tuple' object has no attribute 'append'. Use num[1].append(3).
Merge commit happens because you type a message	It happens because both branches diverged. The message is a side effect.
Merge commit has 3 parents	Merge commit has 2 parents. Commits and parents are different counts.
Wrote 'syntax error', 'none', 'index out of bound'	SyntaxError, None, IndexError: list index out of range. Use the exact name.

4. HOW I IMPROVED TODAY

- Boundary logic and the if/elif chain: correct from the first retest. Reasoning for else pairing improved after one correction.
- Slicing: all 8 outputs correct; reasons got precise once I named the step sign and the stop rule.
- Git: ran the full branch, commit, switch, merge sequence myself and read the graph correctly. Merge commit parent count is fixed after one retest (0, 1, 2).

5. PRACTICE TO RETRY (no solutions)

- Trace: marks = 30; one if/elif/else chain with > 30 and >= 30. Give the output and the reason.
- Write one condition for marks from 30 to 39 using and, then rewrite it as a chained comparison.
- t = (1, [2, 3]); write one line that adds 4 to the list, and one that fails. Name the exact error.
- nums = [10, 20, 30, 40, 50]: predict nums[3:0:-1], nums[-3:], nums[1:99], nums[::2].
- Git, fresh folder: make a diverged merge that gives a conflict (same line, same file), then fix it and read the graph.
- Excel: C2 holds =B2*A1. Copy it to C3. Write the formula, then explain where $ goes if the tax rate sat in a column.

6. RECALL QUESTIONS (answer from memory, no notes)

- What does sort() return, and what does pop() return?
- Why does (5) not make a tuple, and what does?
- What can change in t = (1, [2, 3]) and what cannot? Exact error name?
- How many parents does the first commit, an ordinary commit and a merge commit have?
- What is needed for a fast-forward, and what forces a merge commit?
- Which if does an else belong to? What happens to the inner if when the outer is False?

7. OPEN ITEMS

- Hardest-problem redo from a blank file: not done.
- git push from Colab with a token: not retested.
- Missing $ rule and exact error names: retest once more.